

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

(найменування кафедри)

КУРСОВИЙ ПРОЄКТ

з дисципліни «Конструювання програмного забезпечення»

(назва дисципліни)

на тему: «Застосунок контролю якості винограду»

Студента 3 курсу КНТ-122 групи
спеціальності 121 Інженерія
програмного забезпечення
освітня програма (спеціалізація)
інженерія програмного забезпечення
Онищенко О.А.

(прізвище та ініціали)

Керівник асистент, к.т.н., Леошенко С.
Д.

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна шкала

Кількість балів: Оцінка: ECTS

Члени комісії

Леошенко С. Д.

(підпис)

(прізвище та ініціали)

(підпис)

(прізвище та ініціали)

(підпис)

(прізвище та ініціали)

2025 рік

РЕФЕРАТ

Пояснювальна записка до курсової роботи містить: 71 аркушів, 18 рисунків, 7 таблиць, 4 додатки, 23 джерела.

ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ КОНТРОЛЮ ЯКОСТІ
ВИНОГРАДУ, ВЕБ ЗАСТОСУНОК, DJANGO.

Об'єкт дослідження – контроль якості винограду.

Мета роботи – дослідити предметну область оцінки винограду, зробити програму автоматизації цього процесу.

Методи дослідження – використання відкритих джерел інформації та відомих засобів розробки.

Результатом роботи має бути готова програма, пояснювальна записка, презентація проєкту.

Основні показники:

- Пояснювальна записка згідно стандартів;
- Презентація повністю оформлена і заповнена;
- Програма функціонуюча і зрозуміло написана.

ЗМІСТ

КУРСОВИЙ ПРОЄКТ	1
РЕФЕРАТ	2
ЗМІСТ	3
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ.....	6
ВСТУП.....	7
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ.....	8
1.1 Огляд існуючих методів вирішення завдання.....	8
1.2 Огляд існуючих програмних засобів, що вирішують аналогічні завдання.....	8
1.2.1 Програма Clarisfresh	8
1.2.2 Програма Vidacycle Sectormentor	12
1.2.3 Програма Grape Compass	15
1.2.4 Порівняльна таблиця розглянутих програм	16
1.3 Постановка завдання роботи	16
1.4 Висновки за першим розділом.....	17
2 ВИБІР ТА ОБҐРУНТУВАННЯ СТРУКТУРИ СИСТЕМИ, ЯКА ПРОЄКТУЄТЬСЯ	18
2.1 Вибір інструментарію розробки програмного забезпечення.....	18
2.1.1 Огляд особливостей мови програмування	18
2.1.2 Огляд особливостей обраного середовища розробки	18
2.1.3 Огляд особливостей обраної системи управління базами даних.....	19
2.2 Структурна схема програми	19
2.3 Опис модулів розроблюваної системи.....	20
2.4 Висновки за другим розділом.....	20
3 ОСНОВНІ РІШЕННЯ ЩОДО РЕАЛІЗАЦІЇ КОМПОНЕНТІВ СИСТЕМИ	21
3.1 Алгоритм функціонування розробленої програми.....	21
3.1.1 Алгоритм авторизації	22
3.1.2 Алгоритм додавання партій	22
3.1.3 Алгоритм реєстрації	23
3.2 Вибір шаблону проєктування	23
3.3 Опис модулів розроблюваної системи.....	24
3.4 Проєктування бази даних.....	24
3.5 Висновки за третім розділом	26
4 КЕРІВНИЦТВО ПРОГРАМІСТА	27
4.1 Призначення й умови застосування програми.....	27
4.2 Характеристики програми.....	28

	4
4.3 Звертання до програми	31
4.4 Початкові та вихідні дані	32
4.5 Повідомлення програмісту	32
5 КЕРІВНИЦТВО ОПЕРАТОРА.....	33
5.1 Призначення програми.....	33
5.2 Умови виконання програми.....	33
5.3 Виконання програми.....	33
5.4 Повідомлення оператору.....	33
6 ТЕСТУВАННЯ І РЕЗУЛЬТАТИ	40
6.1 Опис методити проведення тестування.....	40
6.2 Результати використання розробленого програмного забезпечення	40
ВИСНОВКИ	42
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	43
ДОДАТОК А	45
A.1 Вступ	46
A.1.1 Назва програми.....	46
A.1.2 Призначення та область застосування.....	46
A.2 Підстави для розробки.....	46
A.3 Призначення розробки.....	46
A.4 Вимоги до програми	46
A.4.1 Вимоги до функціональних характеристик.....	47
A.4.2 Вимоги до надійності.....	47
A.4.3 Вимоги до технічних та програмних засобів	47
A.5 Вимоги до програмної документації.....	48
A.6 Стадії і етапи розробки.....	48
A.6.1 Стадії розробки.....	48
A.6.2 Етапи розробки	48
A.6.3 Зміст робіт по етапах	49
ДОДАТОК Б.....	50
Б.1 Загальні відомості.....	51
Б.2 Функціональне призначення	51
Б.3 Опис логічної структури програми.....	51
Б.3.1 Алгоритм авторизації.....	51
Б.3.2 Реєстрація	51
Б.3.3 Додавання партії.....	52
Б.4 Використані технічні засоби	52
Б.5 Виклик і завантаження	52
Б.6 Початкові дані.....	53
ДОДАТОК В.....	54

Текст представлення profile:	55
Текст представлення index:	55
Текст представлення bunch:	55
Текст представлення new_bunch:	55
Текст функції process_bunch:	56
Текст функції visualize_bunch:	56
Текст файлу grapes/urls.py:	56
Клас моделі Bunch:	56
Клас форми BunchForm:	57
Текст файлу grapes/admin.py:	57
Текст шаблону base.html:	57
Текст шаблону index.html:	58
Текст шаблону profile.html:	60
Текст шаблону new_bunch.html:	61
Текст шаблону bunch.html:	61
Текст шаблону login.html:	61
Текст шаблону signup.html:	62
Текст класу SignUpView:	62
Текст файлу accounts/urls.py:	62
Текст файлу runner/urls.py:	62
ДОДАТОК Г	63

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

Bootstrap – простий фреймворк стилізації;

Django – бекенд фреймворк для мови Python;

Model-View-Template (Модель-Представлення-Шаблон) – шаблон проєктування фреймворку Django;

MySQL – популярна реляційна база даних;

Python – мова програмування високого рівня.

ВСТУП

Виноград є корисним фруктом. З нього можна виготовляти різні побічні продукти^[1]: родзинки, джем, вино, сік, мармелад, та інші. Виноград росте у гронах і відрізняється кольором, розміром та смаком залежно від виду. Грони винограду ростуть на лозах^[2], для вирощування найкраще підходять сонячні схилісті місцевості на висоті^[3].

Контроль якості винограду краще робити вручну: перебираючи кожную ягоду і дивлячись на колір, розмір, зовнішній вигляд^[4].

За Європейськими стандартами якості виноград можна поділити на три класи^[5]: Екстра, Перший, Другий.

- Клас Екстра не має зовнішніх пошкоджень, має однакову форму і гарну якість;
- Перший клас може мати зовнішні пошкодження і хорошу якість;
- Другий клас може мати значні зовнішні пошкодження і призначений для переробки або на продаж за нижчою ціною.

Для перевірки якості винограду також дивляться на рівень цукру та кислотності^[6].

- Рівень цукру має бути близько 22-30%;
- Рівень кислотності близько 3.2-3.5 pH^[7].

У програмах контролю якості користувач може отримувати зворотній зв'язок в режимі реального часу про результати перевірок. Краще, щоб такі програми давали можливість колаборації: роботи кількох користувачів над одним проєктом, доступ до нього, коментування різних процесів. Через програму контролю якості нові люди в компанії можуть швидше зрозуміти виробничі процеси^[8].

Актуальність програми: може бути корисна для виноградарів і покупців для відстеження якості ягід.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

Програми для контролю якості винограду зазвичай отримують відомості про виноград і на їх основі показують результат. Отримання даних про виноград може бути вручну або автоматизоване. Такі програми також можуть виводити статистику і показувати сповіщення про непройдені тести.

Програми контролю якості будь-чого можуть також на основі провалених тестів показувати пропозиції для покращення. Для програми контролю якості винограду може бути корисним показувати поради як покращити виробництво аби дефектів не було.

1.1 Огляд існуючих методів вирішення завдання

Знайдені приклади є застосунками, що дозволяють використовувати камеру для перевірки зовнішніх пошкоджень ягід або давати інформація про можливі загрози.

В цих програмах користувач може або вручну додавати параметри кольору, розміру, кислотності, або додавати фото, а програма їх аналізуватиме. У будь-якому випадку такі програми в кінці показують результати перевірок і можливо виводять статистику.

1.2 Огляд існуючих програмних засобів, що вирішують аналогічні завдання

1.2.1 Програма Clarisfresh

Програма Clarisfresh є одним з рішень компанії-розробника для перевірки якості продуктів. Застосунок є безкоштовним і дозволяє використовувати камеру для перевірки зовнішніх пошкоджень. Програма

приймає багато параметрів: розмір, колір, зовнішні пошкодження, ушкодження від комах, м'якість. Після введення даних про виноград програма показує результати перевірки та надсилає їх на електронну пошту.

Переваги:

- Простий інтерфейс;
- Можливість додавати фото винограду;
- Отримання результатів на пошту.



Рисунок 1.1 – Головне меню програми^[9]

× **New Inspection** Next
Grapes

Name

Variety

Common Black

Inspection Type

SAP Demo

Standard

Basic Standard

Material

Plant

Supplier

StorageLocation

Temperature

Item weight

Рисунок 1.2 – Застосунок на формі заповнення даних^[10]

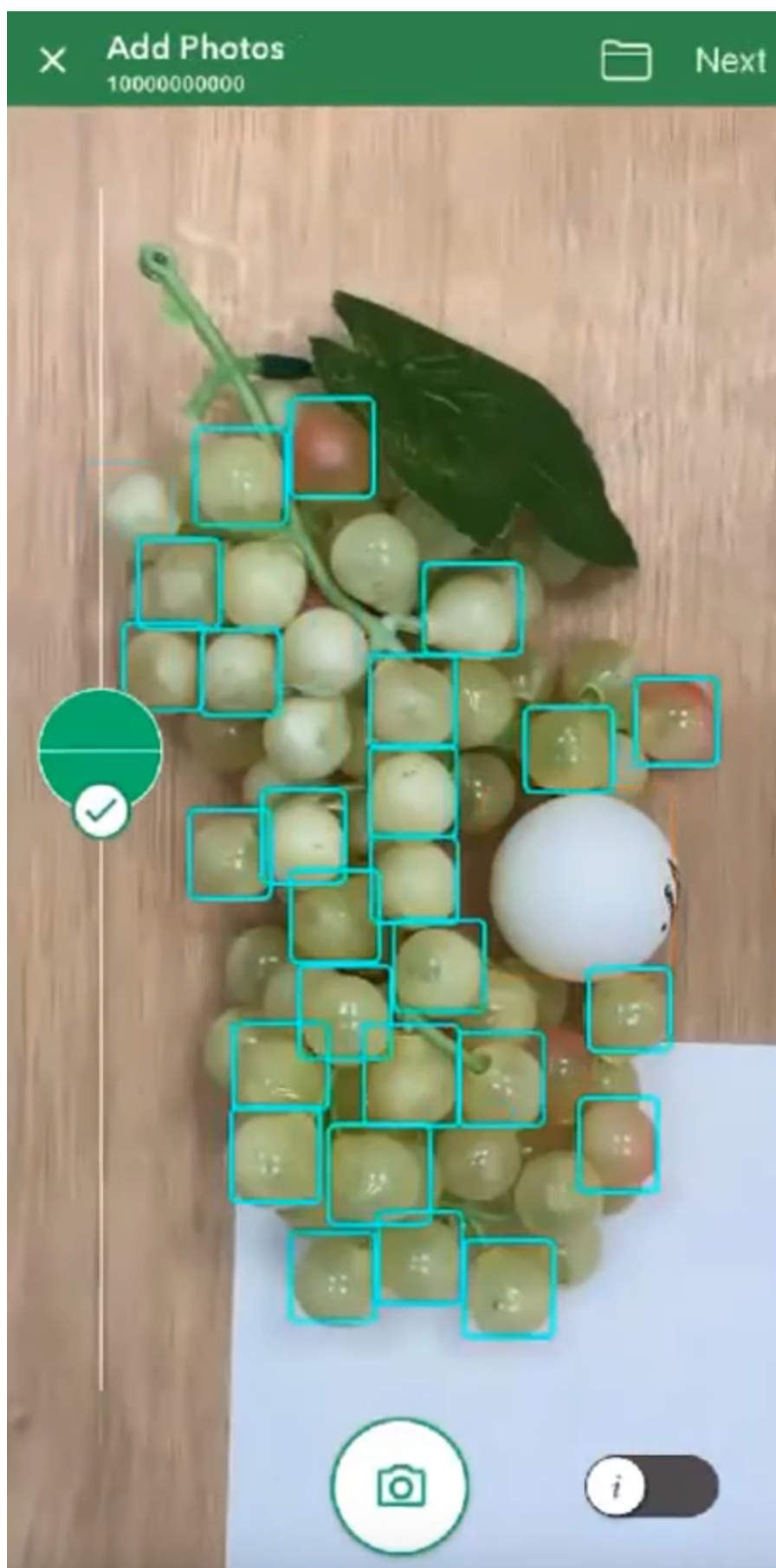


Рисунок 1.3 – Застосунок при додаванні фото винограду^[11]

1.2.2 Програма Vidacycle Sectormentor

Програма Vidacycle Sectormentor є професійним застосунком для полегшення утримання виноградників, отримання звітів про зрілість, стан ґрунту, потенційних проблем, та іншого. Програма є платною і має підходити для ферм будь-якого розміру. Різні версії^[12] програми призначені для виноградників різних розмірів.

Корисними можливостями програми є:

- Звіт роботи^[13]: показує кількість часу витрачену на різні заняття;
- Мапа виноградників^[14]: показує локацію кожного виноградника через GPS;
- Якість ґрунту^[15]: порівнює показники землі з опорними позначками.

Програма також дозволяє вводити дані про виноград через телефон, в тому числі додавати фото.

Переваги:

- Зрозумілий інтерфейс;
- Крос-платформний застосунок;
- Детальні звіти.

Work Report

By Block

Job	Variety	Total	Per ha	Per plant
Pruning (hrs)	Auxerrois	21h 44m	49h 57m	1m 13s
Pruning (hrs)	Bacchus Old	37h 30m	76h 32m	1m 52s
Pruning (hrs)	BacchusNew	51h	91h 4m	2m 4s
Pruning (hrs)	Chardonnay 78	49h	132h 26m	2m 58s
Pruning (hrs)	Chardonnay 95	32h 30m	83h 20m	1m 54s
Pruning (hrs)	Chardonnay 96	39h	86h 40m	1m 56s
Pruning (hrs)	Faber	33h 30m	65h 41m	1m 36s

By Rootstock

Job	Soil	Total	Per ha	Per plant
Pruning (hrs)	clay	139h	80h 21m	1m 57s
Pruning (hrs)	clay / sand	224h 46m	80h 13m	1m 50s
Pruning (hrs)	loam	21h 44m	49h 57m	1m 13s

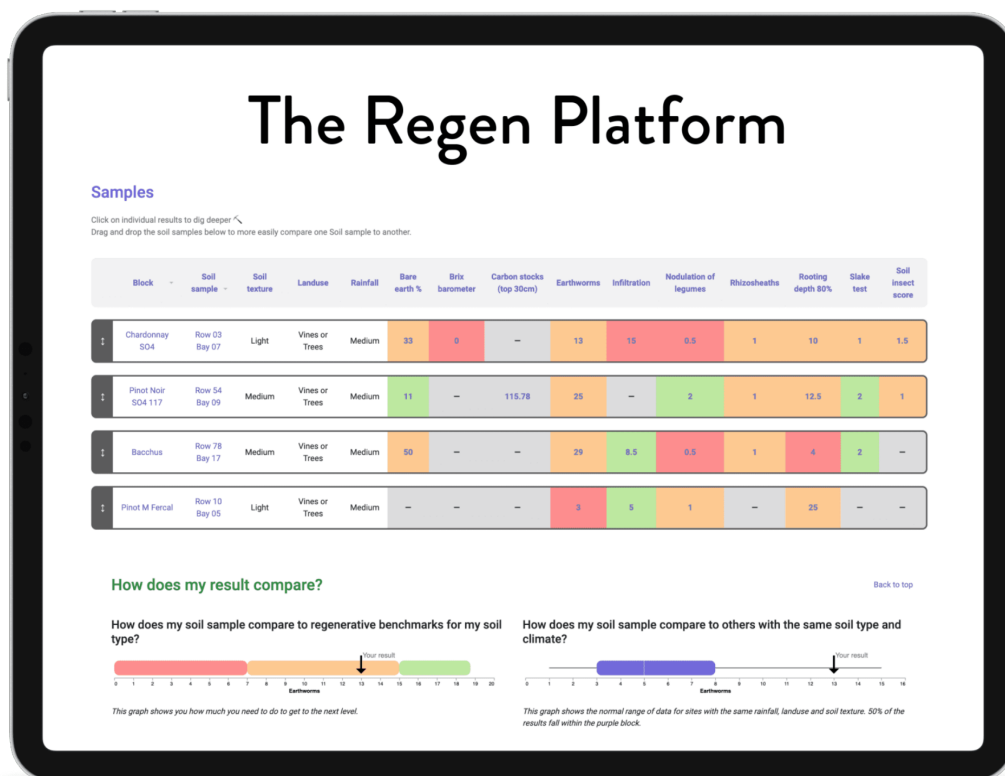
Рисунок 1.4 – Сторінка звіту роботи^[16]Рисунок 1.5 – Сторінка аналізу ґрунту^[18]



Рисунок 1.6 – Форма локацій виноградників^[17]

1.2.3 Програма Grape Compass

Grape Compass є крос-платформним застосунком який попереджає про загрози інфекцій для виноградників. Програма адаптує дані під різні регіони і показує таблички з рівнями загрози для різних днів у різний час.

Переваги:

- Простий інтерфейс;
- Можливість оцінки точності попереджень;
- Крос-платформна програма.

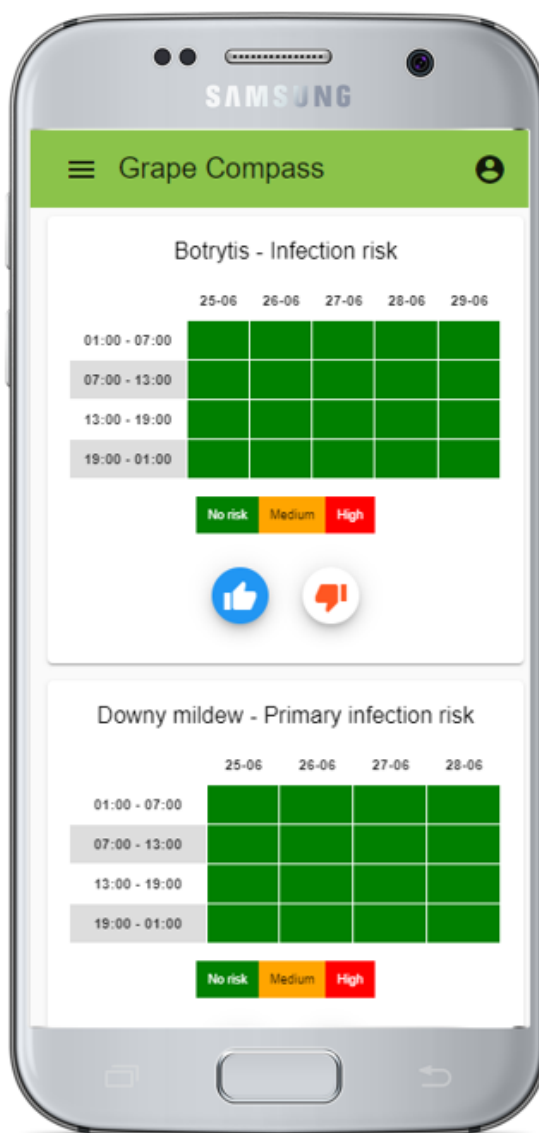


Рисунок 1.7 – Інтерфейс застосунку на телефоні^[19]

1.2.4 Порівняльна таблиця розглянутих програм

Всі три застосунки можуть бути корисні для виноградарників різних розмірів і для різних способів використання.

Програма Grape Compass не підходить для покупців винограду, бо розрахована на власників виноградників. Програма Vidacycle Sectromentor може бути складною для опанування вперше.

Для кращого розуміння розглянутих програм наведено таблицю порівняння:

Таблиця 1.1 - Порівняння прикладів

Критерій	Clarifruit	Vidacycle	Grape Compass
<i>Зручність</i>	Висока	Середня	Висока
<i>Статистика</i>	Базова	Детальна	Проста
<i>Ціна</i>	Безкоштовне	Висока	Безкоштовне
<i>Масштабованість</i>	Середня	Висока	Низька
<i>Платформи</i>	Мобільний	Мобільний та десктоп	Мобільний та веб
<i>Робота офлайн</i>	+	+	-

1.3 Постановка завдання роботи

Мінімально програма має виводити список оброблених ягід та посилання на сторінку додавання нових. Сторінка додавання має питати інформацію про:

- Кількість ягід;
- Зовнішні пошкодження;
- Твердість ягід.

Після обробки сторінка має виводити як результат стадію куди направити виноград.

Після обробки партія має бути додана до списку оброблених ягід.

Кожна оброблена партія повинна мати окрему сторінку з інформацією про неї та результати.

1.4 Висновки за першим розділом

В першому розділі були розглянуті програми допомоги власникам виноградників Clarifruit, Vidacycle Sectormentor, та Grape Compass. Розглянуті програми мають різні рівні складності, бо призначені для різних видів задач.

Актуальність програми доведена існуванням багатьох програм для вирішення цієї задачі.

2 ВИБІР ТА ОБҐРУНТУВАННЯ СТРУКТУРИ СИСТЕМИ, ЯКА ПРОЄКТУЄТЬСЯ

2.1 Вибір інструментарію розробки програмного забезпечення

Для розробки обрано фреймворк Django^[22] через зрозумілість та легку мову програмування. Як база даних має бути використана MySQL^[21] через простоту та знайомість. Для стилізації використано фреймворк Bootstrap^[20] через його зрозумілість.

2.1.1 Огляд особливостей мови програмування

Мовою програмування для Django є Python. Ця мова має простий синтаксис:

- Дозволяє швидко створювати списки;
- Має динамічну типізацію;
- Блоки позначає відступами.

Мова має багато бібліотек.

Таблиця 2.1 - Порівняння інструментів[□]

Критерій	Python, Django	PHP, Laravel	C#, .NET
<i>Зручність мови</i>	Висока	Низька	Середня
<i>Масштабованість</i>	Середня	Низька	Висока
<i>Безпека</i>	Висока	Середня	Вища
<i>Інтерфейс</i>	Шаблони	Фреймворки Blade / Livewire	Фреймворк Blazor

2.1.2 Огляд особливостей обраного середовища розробки

Як середовище розробки використано Visual Studio Code. Воно є простим і має багато розширень.

Таблиця 2.1 – Порівняння середовищ розробки

Критерій	VS Code	Sublime Text	PyCharm
Простота інтерфейсу	Висока	Висока	Середня
Підтримка Python	Висока	Середня	Висока
Підтримка JavaScript	Висока	Середня	Середня

2.1.3 Огляд особливостей обраної системи управління базами даних

База даних MySQL є системою з відкритим кодом. Вона добре масштабується, має багато типів даних. Під’єднання до неї має здійснюватись через пароль та логін.

2.2 Структурна схема програми

Загальна структурна схема на рисунку 2.1.

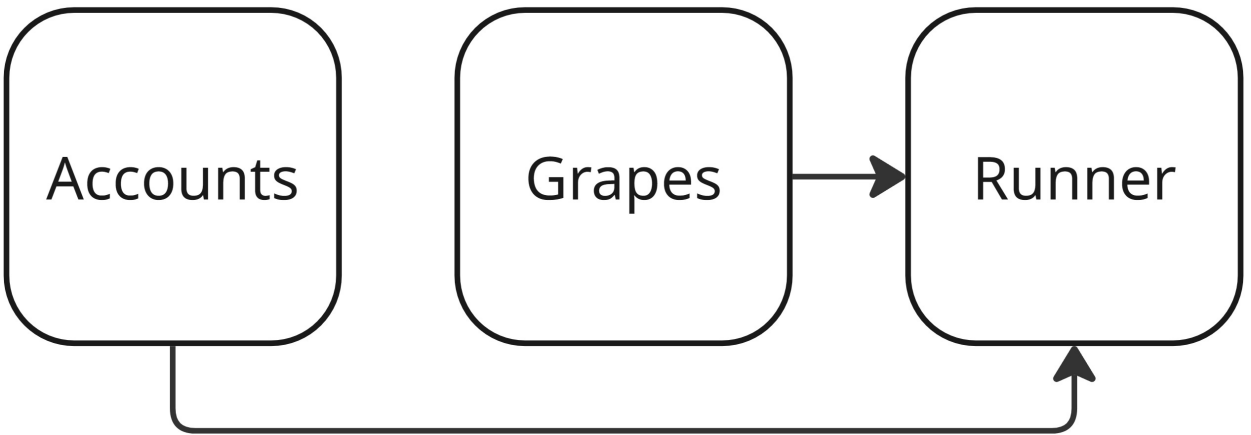


Рисунок 2.1 - Структурна схема програми

2.3 Опис модулів розроблюваної системи

Модуль accounts має відповідати за реєстрацію користувачів. Модуль grapes має містити моделі, представлення і шаблони програми. Модуль runner має зберігати налаштування.

2.4 Висновки за другим розділом

Застосунок має бути розроблювано з використанням фреймворку Django у середовищі VS Code із застосуванням Bootstrap для стилізації та MySQL як базу даних.

3 ОСНОВНІ РІШЕННЯ ЩОДО РЕАЛІЗАЦІЇ КОМПОНЕНТІВ СИСТЕМИ

3.1 Алгоритм функціонування розробленої програми

Можливі алгоритми програми:

- Авторизація;
- Реєстрація;
- Додавання партії.

Процес застосування алгоритмів на рисунку 3.1:

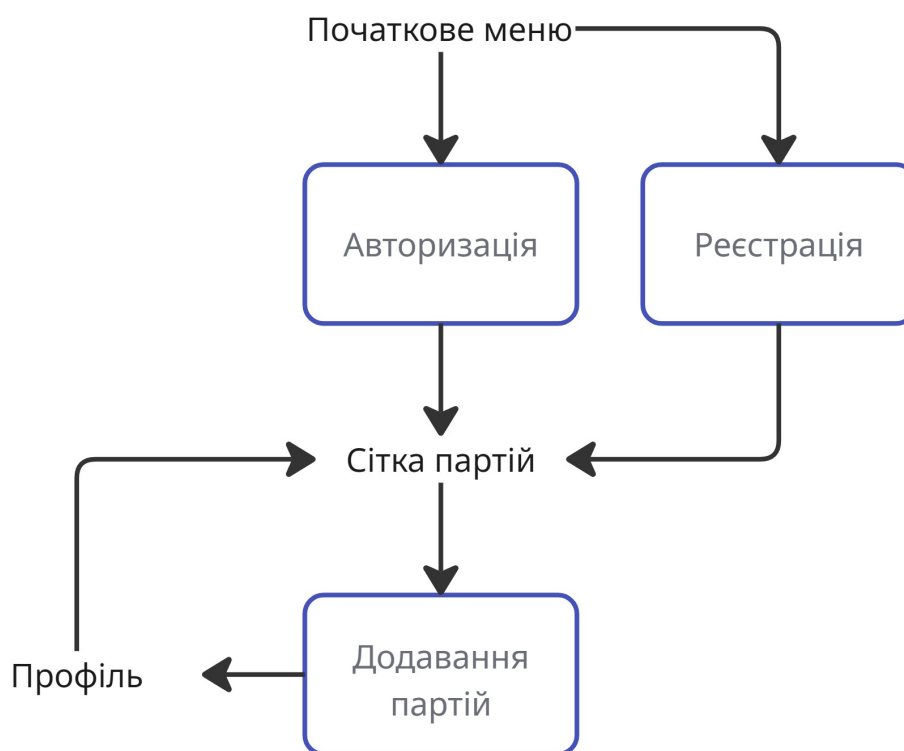


Рисунок 3.1 - Функціональна схема

Програма показує початкове меню з посиланнями на авторизацію та реєстрацію.

3.1.1 Алгоритм авторизації

Якщо користувач зареєстрований в системі, очікує логін (ім'я користувача) та пароль. Програма перевіряє дані в базі і при неправильних видає помилку.^[23]

3.1.2 Алгоритм додавання партій

Сторінка додавання партії очікує:

- кількість винограду;
- статус зовнішніх пошкоджень;
- статус форми;
- статус кольору;
- статус цукру.

Після додавання дані обробляються і додається стадія (куди направити оброблені ягоди):

- Якщо зовнішніх пошкоджень немає; форма, колір, цукор в нормі, та кількість винограду 100 і більше, стадія сік;
- Якщо зовнішніх пошкоджень немає; форма, колір, цукор в нормі, стадія вино;
- Якщо зовнішніх пошкоджень немає; колір, цукор в нормі, стадія продаж;
- Якщо колір та цукор в нормі, стадія родзинки;
- В інших випадках стадія желе.

3.1.3 Алгоритм реєстрації

Сторінка реєстрації очікує ім'я користувача та пароль. Пароль потрібно повторити.

- Ім'я користувача має бути 150 символів і менше;
- Пароль має бути відмінним від імени користувача;
- Пароль має бути 8 символів і більше;
- Пароль не має бути часто використовуваним;
- Пароль не має бути лише числами.

3.2 Вибір шаблону проєктування

Фреймворк Django використовує шаблон Model-View-Template. View обробляє запити та бере дані від Model і відображає їх у Template.

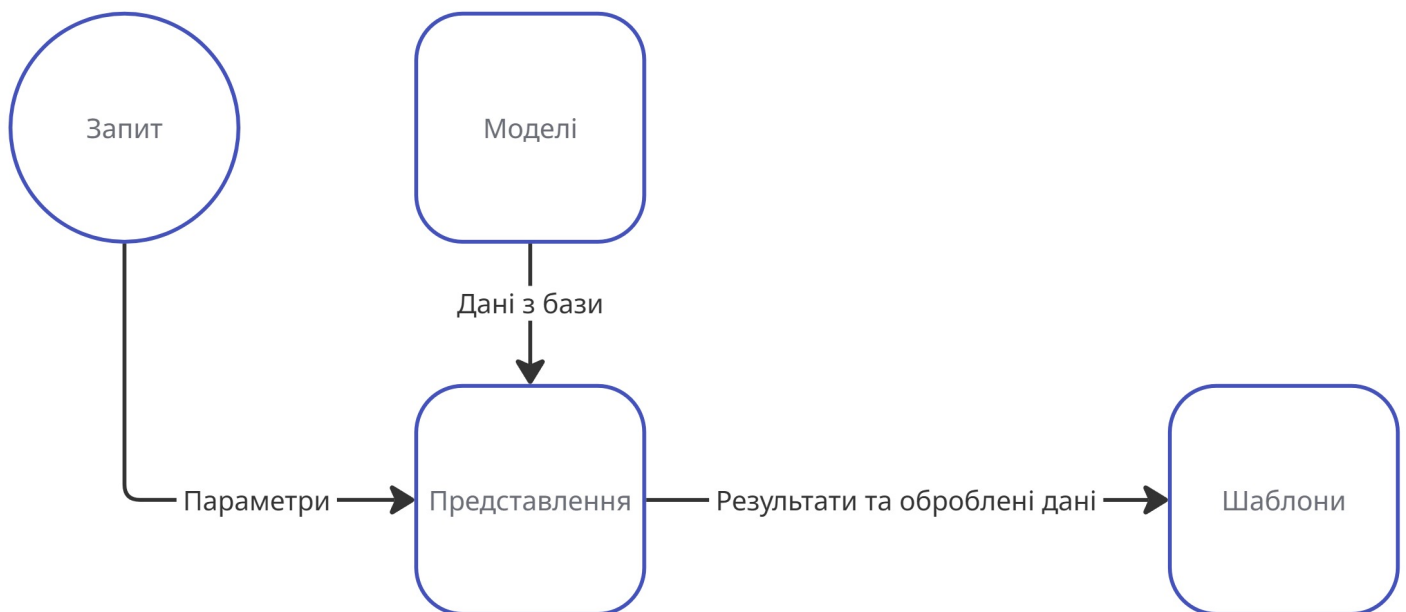


Рисунок 3.2 - Шаблон проєктування

3.3 Опис модулів розроблюваної системи

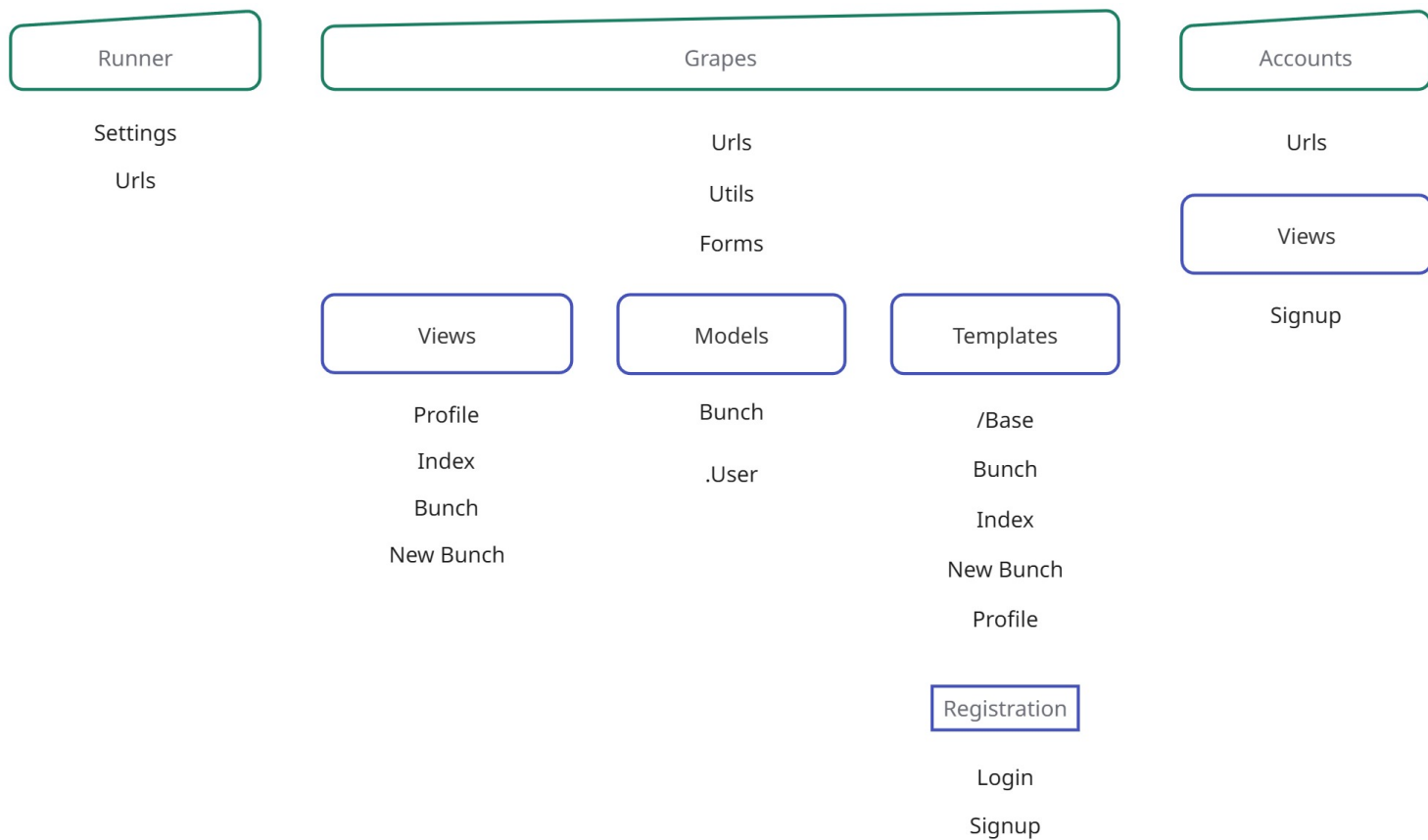


Рисунок 3.3 - Дерево модулів

Програма має складатися з трьох модулів:

- Runner: проєкт з налаштуваннями та глобальними адресами;
- Grapes: основний застосунок з інтерфейсом та моделями;
- Accounts: додатковий застосунок для підлаштування реєстрації;

3.4 Проєктування бази даних

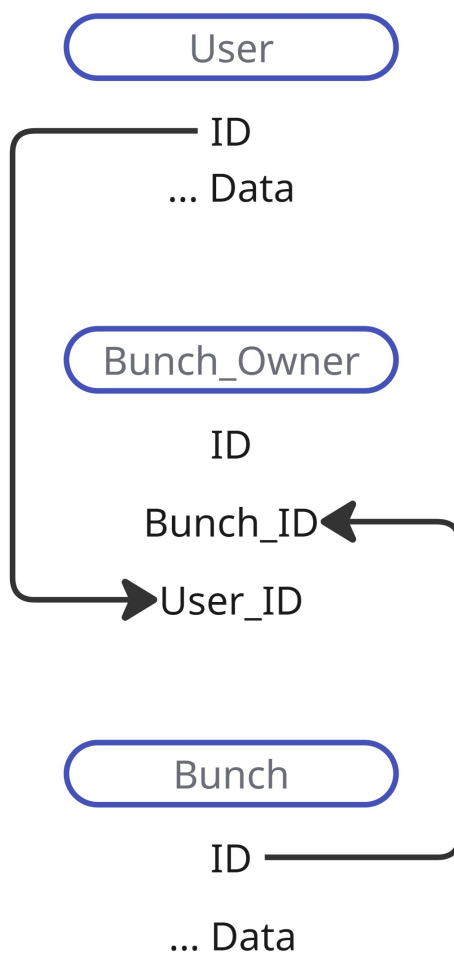


Рисунок 3.4 - Концептуальна модель бази даних

Поле типу TinyInt з розміром 1 є типом Boolean.

Таблиця 3.1 - Опис полів моделі User

Назва	Тип	Розмір	Ключ?	Не порожнє?	Унікальне?
id	int	-	Головний	+	-
password	varchar	128	-	+	-
last_login	datetime	6	-	-	-
is_superuser	tinyInt	1	-	+	-
username	varchar	150	-	+	+
first_name	varchar	150	-	+	-
last_name	varchar	150	-	+	-
email	varchar	254	-	+	-
is_staff	tinyInt	1	-	+	-
is_active	tinyInt	1	-	+	-
date_joined	datetime	6	-	+	-

Всі поля таблиці Bunch є обов'язковими. Розмір проставлено в дужках біля типу.

Таблиця 3.2 - Опис полів моделі Bunch

Назва	Тип	Ключ?
id	bigint	Головний
count	int	-
outside	tinyint(1)	-
sugar	tinyint(1)	-
shape	tinyint(1)	-
color	tinyint(1)	-
stage	varchar(7)	-

Таблиця 3.3 - Опис моделі Bunch_Owner

Назва	Тип	Ключ?
id	bigint	Головний
bunch id	bigint	Зовнішній (Bunch.id)
user id	int	Зовнішній (User.id)

3.5 Висновки за третім розділом

Основні алгоритми в програмі: авторизація, реєстрація та додавання нових партій винограду. Програма приймає запити на представлення, обробляє дані через моделі та виводить на шаблони. Шаблон проектування Model-View-Template є стандартним для Django.

4 КЕРІВНИЦТВО ПРОГРАМІСТА

4.1 Призначення й умови застосування програми

Програма може бути корисна власникам і утримачам виноградників для обліку винограду та невеликої автоматизації виробництва. Вона дозволяє додавати партії зібраного винограду з їх загальними характеристиками та підказує на яку стадію виробництва їх краще відправити. Це забезпечить список вже обробленого винограду і можливу допомогу для обробки доданого.

Програма виконує наступні функції:

- Додавання нової партії;
- Перегляд даних доданих партій;
- Реєстрація користувача;
- Авторизація користувача;
- Зміна кількості стовпців у сітці головного меню;
- Зміна облікового запису користувача;

Для нормального функціонування програми, повинні виконуватись наступні умови до апаратної частини сервера:

- 1 Гб оперативної пам'яті;
- 1 ядро процесору;
- 24 Гб місця на диску.

Для нормального функціонування програми, повинні виконуватись наступні умови до апаратної частини клієнту:

- 4 Гб оперативної пам'яті;
- 2 ядра процесору;
- 3 Гб місця на диску.

Умови до програмної частини, для нормального функціонування програми:

- Windows нових версій;
- Інтегроване середовище розробки;
- Мова Python версії 3.

4.2 Характеристики програми

Програма має таку структуру файлів:

```
.
├── accounts/
│   ├── urls.py
│   └── views.py
├── grapes/
│   ├── templates/
│   │   ├── registration/
│   │   │   ├── login.html
│   │   │   └── signup.html
│   │   ├── base.html
│   │   ├── bunch.html
│   │   ├── index.html
│   │   ├── new_bunch.html
│   │   └── profile.html
│   ├── admin.py
│   ├── forms.py
│   ├── models.py
│   ├── urls.py
│   ├── utils.py
│   └── views.py
├── runner/
│   ├── settings.py
│   └── urls.py
├── .env
├── requirements.txt
└── manage.py
```


1. `accounts/urls.py` додає шлях до представлення реєстрації;
2. `accounts/views.py` містить представлення реєстрації:

Клас `SignUpView` наслідує властивості `CreateView` (показує форму створення об'єкту). Цей клас створює представлення реєстрації користувача на основі вбудованого класу `UserCreationForm`, при успіху перенаправляє на сторінку входу. Для зміни вигляду форми використовує шаблон `templates/registration/signup.html`.

3. `grapes/templates/registration/login.html` показує форму входу;
4. `grapes/templates/registration/signup.html` показує форму реєстрації;
5. `grapes/templates/base.html` оголошує базову структуру сторінок, додає `Bootstrap`;
6. `grapes/templates/bunch.html` показує інформацію про партію;
7. `grapes/templates/index.html` показує початкову сторінку або список партій якщо авторизовано;
8. `grapes/templates/new_bunch.html` показує форму додавання партії;
9. `grapes/templates/profile.html` показує інформацію про користувача;
10. `grapes/admin.py` додає інформацію з моделі `Bunch` до сторінки `/admin`;
11. `grapes/forms.py` містить форму `BunchForm`, яка використовується у `new_bunch` представленні з шаблоном `templates/new_bunch.html`;
12. `grapes/models.py` містить модель `Bunch`:

Модель `Bunch` має:

- 1 зовнішній ключ `owner`, який створює таблицю-посередника в базі даних і з'єднує об'єкти `Bunch` з власниками об'єктами `User`;
- 1 поле цілих чисел `count`, яке зберігає кількість винограду;
- 4 поля булевих значень `outside`, `shape`, `color`, `sugar`, які зберігають інформацію про партію;

- 1 рядкове поле зі значеннями з класу Stage.

Клас Stage зберігає стадії партії після обробки:

- Juice: сік;
- Wine: вино;
- Grapes: виноград на продаж в магазинах;
- Raisins: родзинки;
- Jelly: мармелад.

13. grapes/urls.py містить шляхи до представлень:

- / веде до index;
- /bunch/{bunch.id} веде до bunch;
- /new_bunch/ веде до new_bunch;
- /profile/ веде до profile.

14. grapes/utils.py містить допоміжні функції:

- Функція process_bunch приймає партію типу Bunch і додає стадію;
- Функція visualize_bunch приймає партію типу Bunch і замінює значення її змінних на емоджі для легшого сприйняття.

15. grapes/views.py містить представлення:

- profile отримує поточного користувача, його партії та виводить шаблон profile;
- index отримує партії поточного користувача і виводить шаблон index;
- bunch отримує партію за ідентифікатором, візуалізує її через visualize_bunch і виводить шаблон bunch;

- `new_bunch` виводить форму `BunchForm`: якщо запит на отримання сторінки то виводить шаблон `new_bunch`, якщо запит на відправлення форми то обробляє дані через `process_bunch`, додає власника і зберігає об'єкт в базі. При успішному надсиланні форми переадресовує на сторінку партії.

16. `runner/settings.py` зберігає глобальні налаштування, в тому числі бази даних MySQL, налаштування Bootstrap та Crispy Forms (додаток який дозволяє використовувати Bootstrap у формах Django);

17. `runner/urls.py` основний файл з шляхами:

- `/` веде до шляхів `grapes/urls.py`;
- `/accounts/` веде до вбудованих шляхів `django.contrib.auth.urls` (вони визначають шляхи автентифікації), а також до шляхів `accounts/urls.py`;

18. `.env` зберігає облікову інформацію для бази даних MySQL;

19. `requirements.txt` зберігає список використаних сторонніх бібліотек: `django`, `mysqlclient`, `python-dotenv`, `django-crispy-forms`, `crispy-bootstrap5`;

20. `manage.py` використовується для запуску застосунку з терміналу.

4.3 Звертання до програми

В терміналі можна використати такі команди:

- `py manage.py runserver` - запуск серверу розробки;
- `py manage.py makemigrations назваЗастосунку (grapes або accounts)` - створити зміни до бази даних запитом SQL;
- `py manage.py migrate` - застосувати зміни до бази даних;
- `django-admin startproject назваПроекту шляхДоТеки` - створити новий проєкт;

- `py manage.py startapp назваЗастосунку` - додати новий додаток до проєкту.

4.4 Початкові та вихідні дані

Програма створює в базі даних таблиці, з яких використовуються:

- `auth_user` - дані про користувачів;
- `grapes_bunch` - дані про об'єкти Bunch;
- `grapes_bunch_owner` - зв'язкує користувачів з партіями.

4.5 Повідомлення програмісту

При запуску робочого серверу в консоль виводиться:

- Перевірки помилок;
- Дата та час;
- Версія Django;
- Посилання робочого серверу;
- Попередження.

При кожному запиті до серверу в консоль виводиться тип запиту, посилання запиту, код відповіді.

5 КЕРІВНИЦТВО ОПЕРАТОРА

5.1 Призначення програми

Програма може полегшити вести облік партій і їх обробку. Після додавання даних про партію програма додає рекомендовану стадію. Додані партії можна переглянути у профілі.

5.2 Умови виконання програми

Програма виконується у браузері і підходить як для мобільних так і для десктопних пристроїв.

5.3 Виконання програми

Для запуску програми потрібно:

- Завантажити проєкт на комп'ютер;
- В терміналі написати: `.venv\scripts\activate, py manage.py runserver;`
- Перейти на `http://127.0.0.1:8000/`.

5.4 Повідомлення оператору

Інтерфейс програми нижче:



Рисунок 5.1 - Початкове меню

Grapes Checker

Profile

Username*

Password*

Log in

Рисунок 5.2 - Сторінка входу

Grapes Checker

Profile

Username*

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password*

- Your password can't be too similar to your other personal information.
- Your password must contain at least 8 characters.
- Your password can't be a commonly used password.
- Your password can't be entirely numeric.

Password confirmation*

Enter the same password as before, for verification.

Sign up

Рисунок 5.3 - Сторінка реєстрації

Grapes Checker

Profile

This is seesm's profile.

Log out

Bunches processed: 7

	Number of grapes	Is outside damaged?	Is shape unstable?	Is colo unevei
11.	52			
13.	38			
14.	73			
15.	175			
22.	74			
23.	35			
24.	38			

Add new bunch

Рисунок 5.4 - Сторінка профілю

Grapes Checker

Profile

Count*

0

⌵

☐

Outside

☐

Shape

☐

Color

☐

Sugar

Add bunch

Рисунок 5.5 - Сторінка додавання нової партії

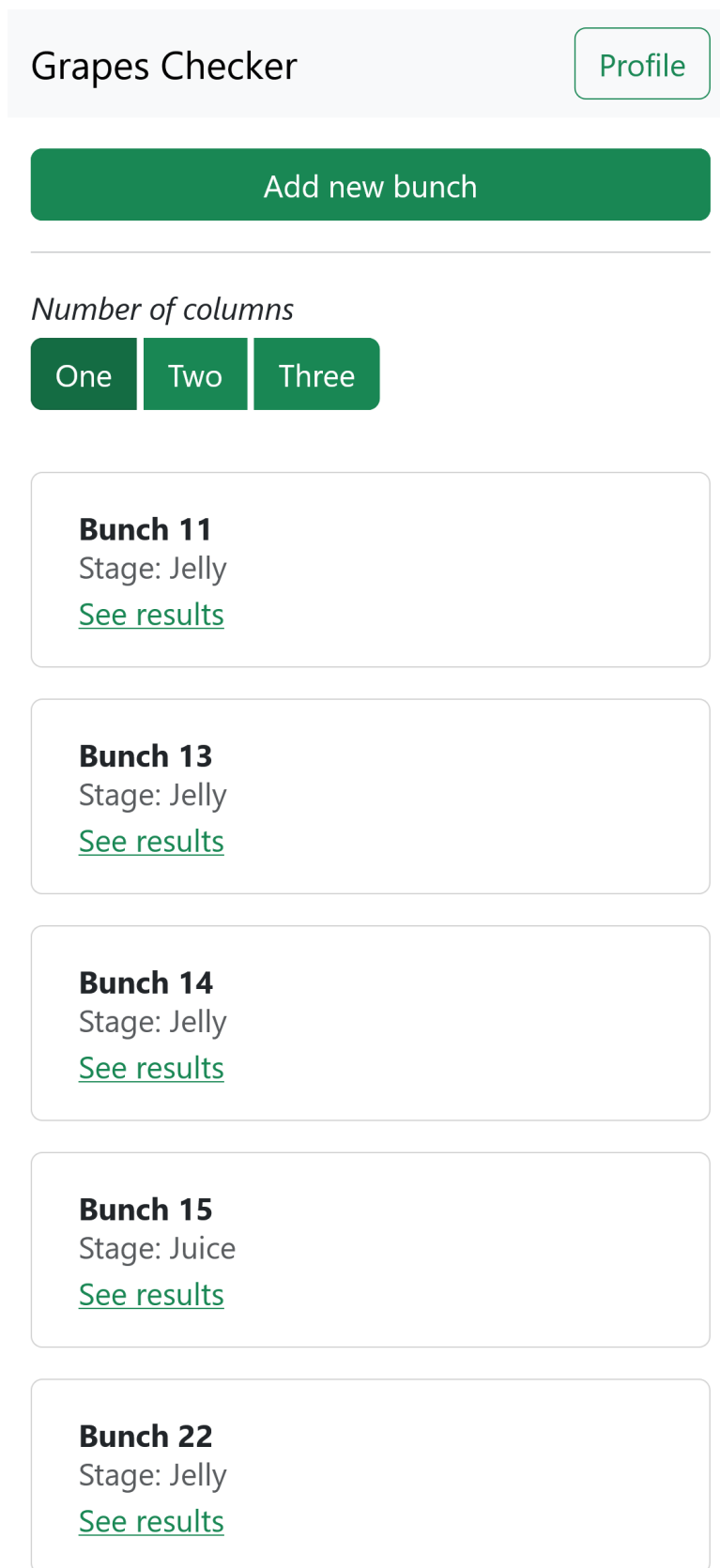


Рисунок 5.6 - Головне меню

6 ТЕСТУВАННЯ І РЕЗУЛЬТАТИ

6.1 Опис методити проведення тестування

Тестування проведено за таким сценарієм:

1. Гість заходить на сайт
2. Реєструється з іменем guest_test
3. Заходить у кабінет
4. Створює партію з параметрами count=12, outside=True, shape=True, color=False, sugar=False
5. Переглядає партію на сторінці доданої партії
6. Переглядає партію в головному меню
7. Переглядає партію в особистому кабінеті

Успішними результатами тестування мають бути:

- Користувача зареєстровано
- Користувач успішно увійшов у кабінет
- Партію додано з правильними параметрами
- Стадія партії - Raisins (родзинки)
- Дані партії всюди показуються правильно

6.2 Результати використання розробленого програмного забезпечення

Таблиця 6.1 - Результат виконання тестів

Назва	Результат
Вхід на сайт	Успіх
Реєстрація	Успіх

Вхід у кабінет	Успіх
Створення партії	Успіх
Перегляд партії на її сторінці	Успіх
Перегляд партії в кабінеті	Успіх
Перегляд партії в меню	Успіх

ВИСНОВКИ

У ході виконання курсового проєкту, проведено детальне ознайомлення із предметною областю – збирання та обробка винограду. Ознайомлення із найбільш важливими питаннями цієї предметної області та найвагомішими рішеннями: збором та оцінкою якості винограду та застосунками прикладами Clarifresh, Sectormentor, Grape Compass.

Під час роботи із першим розділом було оглянуто принцип роботи програм контролю якості, зокрема і винограду. Також було розглянуто особливості програм прикладів та їх інтерфейси.

У другому розділі розглянуто мову програмування Python та фреймворк Django. Їх було порівняно з мовою PHP та фреймворком Laravel і мовою C# та фреймворком .NET. Також розглянуто структуру програми та її загальні модулі.

У третьому розділі оглянуто алгоритми програми. Шаблон проєктування Model-View-Template є стандартним для фреймворку Django. Також розглянуто таблиці бази даних. Система керування базою даних обрано MySQL.

В решті-решт, було зроблено програму, яка реалізує всі поставлені задачі та список функціональних вимог: система авторизації користувачів, додавання партій та їх аналіз.

Тестування програми довело її працездатність. Помилки при виконанні не виявлено, але програма не є готовим продуктом через брак функціоналу.

Надалі можна модернізувати програму за допомогою використання Tailwind для стилізації, Qwik для фронтенду та розміщення програми на сервері.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. [Grape fruit products | Raisins, Grape wine, juice, vinegar, sweet spreads, grape butter, jelly](https://www.fruitsinfo.com/grape-products.php) [Electronic resource]. – Access mode: <https://www.fruitsinfo.com/grape-products.php>
2. [How to grow grapes / RHS](https://www.rhs.org.uk/fruit/grapes/grow-your-own) [Electronic resource]. – Access mode: <https://www.rhs.org.uk/fruit/grapes/grow-your-own>
3. [What does altitude matter? | Matthew Clark](https://www.matthewclark.co.uk/latest-news-blogs/blog/what-does-altitude-matter/) [Electronic resource]. – Access mode: <https://www.matthewclark.co.uk/latest-news-blogs/blog/what-does-altitude-matter/>
4. [Grapes Quality Control Solution by Clarifresh](https://clarifresh.com/fresh-produce/grapes/) [Electronic resource]. – Access mode: <https://clarifresh.com/fresh-produce/grapes/>
5. [Grapes Safety Testing: Pesticides, Contaminants, & Quality Assurance](https://www.eurofins.in/food-testing/industries/grapes-testing) [Electronic resource]. – Access mode: <https://www.eurofins.in/food-testing/industries/grapes-testing>
6. [How to evaluate the quality of wine grapes before you buy? - Bay Area California wine grapes for home winemakers](https://www.baywinebrokers.com/how-to-evaluate-the-quality-of-wine-grapes-before-you-buy/) [Electronic resource]. – Access mode: <https://www.baywinebrokers.com/how-to-evaluate-the-quality-of-wine-grapes-before-you-buy/>
7. [pH — Вікіпедія](https://uk.wikipedia.org/wiki/PH) [Електронний ресурс]. – Режим доступу: <https://uk.wikipedia.org/wiki/PH>
8. [What is Quality Control Management Software? | QIMAone](https://www.qimaone.com/resource-hub/what-is-a-quality-control-software) [Electronic resource]. – Access mode: <https://www.qimaone.com/resource-hub/what-is-a-quality-control-software>
9. [Clarifruit - Step into the Future of Fresh Fruit and Vegetable Quality Control](https://youtu.be/hNli7jHEd9I?t=302) [Electronic resource]. – Access mode: <https://youtu.be/hNli7jHEd9I?t=302>
10. [Clarifruit - Step into the Future of Fresh Fruit and Vegetable Quality Control](https://youtu.be/hNli7jHEd9I?t=324) [Electronic resource]. – Access mode: <https://youtu.be/hNli7jHEd9I?t=324>

11. [Clarifruit - Step into the Future of Fresh Fruit and Vegetable Quality Control](https://youtu.be/hNli7jHEd9I?t=346) [Electronic resource]. – Access mode: <https://youtu.be/hNli7jHEd9I?t=346>
12. [Buy – Sctormentor](https://vines.vidacycle.com/buy/) [Electronic resource]. – Access mode: <https://vines.vidacycle.com/buy/>
13. [Work Report – Sctormentor](https://vines.vidacycle.com/monitoring-guide/sctormentor-tools-work-report/) [Electronic resource]. – Access mode: <https://vines.vidacycle.com/monitoring-guide/sctormentor-tools-work-report/>
14. [Map your Vineyard – Sctormentor](https://vines.vidacycle.com/monitoring-guide/map-your-vineyard/) [Electronic resource]. – Access mode: <https://vines.vidacycle.com/monitoring-guide/map-your-vineyard/>
15. [The Regen Platform – Sctormentor](https://vines.vidacycle.com/monitoring-guide/the-regen-platform/) [Electronic resource]. – Access mode: <https://vines.vidacycle.com/monitoring-guide/the-regen-platform/>
16. [Work Report Sample](https://vines.vidacycle.com/wp-content/uploads/2021/09/Sctormentor-Work-Report-M-U-1536x1201.png) [Electronic resource]. – Access mode: <https://vines.vidacycle.com/wp-content/uploads/2021/09/Sctormentor-Work-Report-M-U-1536x1201.png>
17. [Wineyard Mapper Sample](https://vines.vidacycle.com/wp-content/uploads/2022/08/Group-18.png) [Electronic resource]. – Access mode: <https://vines.vidacycle.com/wp-content/uploads/2022/08/Group-18.png>
18. [Regen Platform Sample](https://vines.vidacycle.com/wp-content/uploads/2023/11/Soilmentor-Regen-Platform-M-U-1920x1352.png) [Electronic resource]. – Access mode: <https://vines.vidacycle.com/wp-content/uploads/2023/11/Soilmentor-Regen-Platform-M-U-1920x1352.png>
19. [Grape Compass Interface](https://www.grapecompass.com/wp-content/uploads/sites/9/2021/10/GrapeCompass_app_Samsung.png) [Electronic resource]. – Access mode: https://www.grapecompass.com/wp-content/uploads/sites/9/2021/10/GrapeCompass_app_Samsung.png
20. [MySQL](https://www.mysql.com/) [Electronic resource]. – Access mode: <https://www.mysql.com/>
21. [Bootstrap](https://getbootstrap.com/) [Electronic resource]. – Access mode: <https://getbootstrap.com/>
22. [Django](https://www.djangoproject.com/) [Electronic resource]. – Access mode: <https://www.djangoproject.com/>
23. [Django Login, Logout, Signup, Password Change, and Password Reset | LearnDjango.com](https://learndjango.com/tutorials/django-login-and-logout-tutorial) [Electronic resource]. – Access mode: <https://learndjango.com/tutorials/django-login-and-logout-tutorial>

ДОДАТОК А

Технічне завдання

A.1 Вступ

A.1.1 Назва програми

Програма має назву «Застосунок для контролю якості винограду».

A.1.2 Призначення та область застосування

Програма призначена для перевірки винограду за його зовнішніми і внутрішніми параметрами та виведення результатів.

Програма може використовуватися власниками виноградників для спрощення ланцюжку постачання і економії часу на обробці винограду.

A.2 Підстави для розробки

Підставою розробки програми є створення застосунку, що має полегшити перевірку якості винограду на виробництві.

A.3 Призначення розробки

Програма має дозволяти користувачам надавати інформацію про виноград та на її основі виводити результати перевірок.

Це може бути корисним для швидшої оцінки і огляду ягід для їх подальшого сортування.

A.4 Вимоги до програми

Основними вимогами до програми є:

- Клієнт-серверна архітектура;
- Інтерактивний інтерфейс.

А.4.1 Вимоги до функціональних характеристик

Програма має виконувати такі мінімальні функції:

- Введення даних про виноград;
- Аналіз даних та проходження перевірок;
- Виведення результатів користувачеві.

Такі вимоги до функціональності мають забезпечити інтерактивність інтерфейсу.

Для виконання вимоги клієнт-серверної архітектури застосунок має використовувати віддалену базу даних, можливо MongoDB.

А.4.2 Вимоги до надійності

Програма має містити обробники виключних ситуацій аби не допустити непланового завершення роботи.

Для надійності даних програма має дозволяти зберігати їх на базі даних і отримувати за потребою. Доступ до бази даних має бути обмежений паролем.

А.4.3 Вимоги до технічних та програмних засобів

Програма має запускатися у браузері і бути доступною як на мобільних, так і на домашніх комп'ютерах.

- Процесор: x64 / x86 / ARM;
- Оперативна пам'ять: 512 МБ;
- Накопичувач: 1 ГБ вільного місця;
- Операційна система: Android, iOS, macOS, Linux, Windows.

Програмні засоби можуть змінюватись в ході розробки, але заплановано використати такі:

- Django: фреймворк бекенду;
- MongoDB: база даних;
- Vue.js / React.js: фреймворк фронтенду.

Для користувацької частини (фронтенду) застосунок може використовувати шаблони Django замість фреймворку.

Як база даних також може бути використана MySQL.

A.5 Вимоги до програмної документації

Програмна документація має бути реалізована у:

- Пояснювальній записці;
- Технічному завданні.

Код програми має бути написаний із зрозумілими назвами змінних/класів/методів і може включати коментарі.

A.6 Стадії і етапи розробки

A.6.1 Стадії розробки

Стадії розробки:

1. Розробка документації;
2. Написання програми;
3. Розробка презентації.

A.6.2 Етапи розробки

Етапи розробки:

1. Вибір технічних засобів;
2. Ітеративне написання програми;
3. Тестування та рефакторинг.

А.6.3 Зміст робіт по етапах

Зміст роботи може бути таким на кожному етапі:

1. Початкова документація, тестові програми;
2. Робоча програма, більш заповнена документація, підготовка до презентації;
3. Готова програма, готова документація, готова презентація.

ДОДАТОК Б

Опис програми

Б.1 Загальні відомості

Назва програми: застосунок контролю якості винограду». Мета: полегшення виробництва та обліку винограду. Кінцеві користувачі: виноградарі або люди, що доглядають виноградники. Платформа: веб-браузер. Інтерфейс підходить для: мобільних та настільних комп'ютерів.

Б.2 Функціональне призначення

Реєстрація користувачів, авторизація користувачів, зміна користувача, додавання партій винограду, вказання параметрів партії, перегляд партій в меню, профілі та на окремій сторінці.

Б.3 Опис логічної структури програми

Три рівні: шаблони, представлення, моделі: користувач вводить адресу сайту в браузері. Цей запит надходить на представлення. Воно отримує потрібні дані через моделі, обробляє їх допоміжними функціями та передає у шаблон. Шаблон показує дані у вигляді сторінки.

Б.3.1 Алгоритм авторизації

Програма виводить сторінку з формою. У ній від користувача очікується ім'я та пароль. Після надсилання дані перевіряються в базі. Якщо вони правильні, переадресує на головну сторінку.

Б.3.2 Реєстрація

Сторінка реєстрації очікує ім'я користувача та пароль два рази. Ім'я користувача має бути унікальним і не довгим. Пароль не має бути схожим на

ім'я користувача, бути не менше 8 символів, не мати лише чисел, не бути розповсюдженим. Якщо форму заповнено правильно, користувача переносить до сторінки входу. Якщо є помилки, їх виводить в інтерфейсі.

Б.3.3 Додавання партії

Від користувача очікується кількість винограду та статуси зовнішніх пошкоджень, пошкоджень кольору, форми та нестабільного цукру. Після додавання програма створює об'єкт в базі даних і додає рекомендовану стадію.

Б.4 Використані технічні засоби

Для розробки використано мову Python та фреймворк Django. Мову обрано через простоту синтаксису, наявність бібліотек, знайомість. Фреймворк обрано через хорошу документацію, вбудовані функції та простоту використання.

Аби розгорнути систему, її код потрібно завантажити на сервер. Для серверу достатньо 1 Гб оперативної пам'яті, 1 ядра процесору та 4 Гб оперативної пам'яті. На сервері має бути встановлено Python версії 3.

Для клієнту потрібно 2 або 4 Гб оперативної пам'яті, 2 ядра процесора, 8 Гб оперативної пам'яті. На клієнті має бути встановлено веб-браузер.

Б.5 Виклик і завантаження

Для розгорнення серверу потрібно:

1. Завантажити код програми на свою систему;

2. В консолі активувати віртуальне середовище (`.venv\scripts\activate`) та запустити робочий сервер (`py manage.py runserver`).

Для запуску програми в рядку браузеру ввести адресу `http://127.0.0.1:8000`.

Б.6 Початкові дані

Програма використовує базу даних MySQL. В ній створюються таблиці, зокрема `django_user`, `django_bunch`, `django_bunch_owner`. При роботі можуть додаватися користувачі до `django_user`, об'єкти партій до `django_bunch` та зв'язки між ними та користувачами до `django_bunch_owner`. Дані про сесії також додаються до `django_session`.

ДОДАТОК В

Текст програми

Текст представления profile:

```
@login_required
def profile(request):
    user=request.user
    bunches=Bunch.objects.filter(owner=request.user)
    return render(request, "profile.html", {"user": user, "bunches": bunches})
```

Текст представления index:

```
def index(request):
    all_bunches=list()
    if request.user.is_authenticated:
        all_bunches=Bunch.objects.filter(owner=request.user)
        for bunch in all_bunches:
            processed_bunch=process_bunch(bunch)
            bunch.stage=processed_bunch.stage
            bunch.save()
    return render(request, "index.html", {"bunches": all_bunches, "user":
request.user})
```

Текст представления bunch:

```
@login_required
def bunch(request, id):
    bunch=Bunch.objects.get(pk=id)
    visual_bunch=visualize_bunch(bunch)
    return render(request, "bunch.html", {"bunch": visual_bunch})
```

Текст представления new_bunch:

```
@login_required
def new_bunch(request):
    if request.method=="POST":
        form=BunchForm(request.POST)
        if form.is_valid():
            data=form.cleaned_data
            # create object
            bunch=Bunch(
                count=data['count'],
                outside=data['outside'],
                sugar=data['sugar'],
                shape=data['shape'],
                color=data['color'],
            )
            # process data and make stage
            processed_bunch=process_bunch(bunch)
            processed_bunch.save()
            # add owner
            bunch.owner.add(request.user)
            processed_bunch.save()
```

```

        return HttpResponseRedirect(f'/bunch/{processed_bunch.id}')
    else:
        form=BunchForm()

    return render(request,"new_bunch.html",{"form":form})

```

Текст функції process_bunch:

```

def process_bunch(b: Bunch):
    if not b.outside and not b.shape and not b.color and not b.sugar and
    b.count>=100:
        b.stage=Bunch.Stage.Juice
    elif not b.outside and not b.shape and not b.color and not b.sugar:
        b.stage=Bunch.Stage.Wine
    elif not b.outside and not b.color and not b.sugar:
        b.stage=Bunch.Stage.Grapes
    elif not b.color and not b.sugar:
        b.stage=Bunch.Stage.Raisins
    else:
        b.stage=Bunch.Stage.Jelly

    return b

```

Текст функції visualize_bunch:

```

def visualize_bunch(b: Bunch):
    OK="емоджіПалецьВгору"
    NO="емоджіПалецьВниз"

    b.outside=OK if not b.outside else NO
    b.shape=OK if not b.shape else NO
    b.color=OK if not b.color else NO
    b.sugar=OK if not b.sugar else NO

    return b

```

Текст файлу grapes/urls.py:

```

app_name = 'grapes'
urlpatterns = [
    path('',views.index,name='index'),
    path('bunch/<int:id>',views.bunch,name='bunch'),
    path('new_bunch',views.new_bunch,name='new_bunch'),
    path('profile',views.profile,name='profile'),
]

```

Клас моделі Bunch:

```

class Bunch(models.Model):
    owner=models.ManyToManyField(User)
    count=models.IntegerField(default=0)
    outside=models.BooleanField(default=False)

```

```

shape=models.BooleanField(default=False)
color=models.BooleanField(default=False)
sugar=models.BooleanField(default=False)

class Stage(models.TextChoices):
    Juice='Juice'
    Wine='Wine'
    Grapes='Grapes'
    Raisins='Raisins'
    Jelly='Jelly'
stage=models.CharField(max_length=7,choices=Stage,blank=True)

def __str__(self):
    return f'{self.count} grapes: outside {self.outside}, shape {self.shape},
color {self.color}, sugar {self.sugar}. Stage: {self.stage}'

```

Клас форми BunchForm:

```

class BunchForm(ModelForm):
    class Meta:
        model=Bunch
        fields=('count','outside','shape','color','sugar')

```

Текст файлу grapes/admin.py:

```

class BunchAdmin(admin.ModelAdmin):
    model=Bunch
    list_display=['count','outside','shape','color','sugar','stage']

admin.site.register(Bunch,BunchAdmin)

```

Текст шаблону base.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>{% block title %}{% endblock title %}</title>

    {% comment %} Bootstrap includes {% endcomment %}
    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.5/dist/css/bootstrap.min.
css"
        rel="stylesheet"
        integrity="sha384-
Sg0Ja3DmI69IUzQ2PVdRZhwQ+dy64/BUtBMJw1MZ8t5HZApCHrRKUc4W0kG879m7"
        crossorigin="anonymous"
    />
    <script
        defer
        src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.5/dist/js/bootstrap.bundle
.min.js"

```

```

    integrity="sha384-
k6d4wzSIapyDyv1kpU366/PK5hCdSbCRGRCMv+ep10QJWyd1fbcAu90CUj5zNLiq"
    crossorigin="anonymous"
  ></script>
</head>
<body>
  <nav class="navbar bg-body-tertiary">
    <div class="container">
      <a href="{% url 'grapes:index' %}" class="navbar-brand">Grapes
Checker</a>
      <a href="{% url 'grapes:profile' %}" class="btn btn-outline-
success">Profile</a>
    </div>
  </nav>
  <div class="container my-3">
    {% block content %}{% endblock content %}
  </div>
  <script type="module">
    {% block code %}{% endblock code %}
  </script>
</body>
</html>

```

Текст шаблоны index.html:

```

{% extends 'base.html' %}

{% block title %}
Grapes app
{% endblock %}

{% block content %}
{% if user.is_authenticated %}
<div class="row row-cols-1">
<div class="col">
  <a id="newBunch" style="width: 100%;" class="btn btn-success col" href="{%
url 'grapes:new_bunch' %}">Add new bunch</a>
</div>

<div class="col">
  <hr class="mt-3">
  <em>Number of columns</em>
  <div style="display: block;" class="btn-group mt-1" role="group">
    <button id="one" type="button" class="btn btn-success">One</button>
    <button id="two" type="button" class="btn btn-success">Two</button>
    <button id="three" type="button" class="btn btn-success">Three</button>
  </div>
</div>
</div>

<div style="padding: 0 .5rem;" class="my-3 row row-cols-1 g-3"
id="gridContainer">
  {% if bunches %}
  {% for bunch in bunches %}
    <div class="card col-50">
      <div class="card-body">
        <strong class="card-title">Bunch {{ bunch.id }}</strong>
        <p class="card-subtitle text-body-secondary">Stage: {{ bunch.stage
}}</p>

```

```

    <a class="card-link text-success" href="{% url 'grapes:bunch' bunch.id
%}">See results</a>
  </div>
</div>
{% endfor %}
{% else %}
  <p>No grapes found</p>
{% endif %}
</div>
{% else %}
<h3>This is grapes quality control app.</h3>

<a href="{% url 'login' %}" class="btn btn-success mt-3">Log in</a>
<a href="{% url 'signup' %}" class="btn btn-primary mt-3">Don't have an
account?</a>
{% endif %}
{% endblock %}

{% block code %}
const activeButton=1;
const gridContainer=document.querySelector("#gridContainer");

const oneButton=document.querySelector("#one");
const twoButton=document.querySelector("#two");
const threeButton=document.querySelector("#three");

const activateButton={()=>{
  if (activeButton===1) oneButton.classList.add("active");
  else if (activeButton===2) twoButton.classList.add("active");
  else if (activeButton===3) threeButton.classList.add("active");
}}

const cleanContainer = ()=>{
  gridContainer.classList.remove("row-cols-3");
  gridContainer.classList.remove("row-cols-2");
  gridContainer.classList.remove("row-cols-1");
}

const removeActiveButton={()=>{
  oneButton.classList.remove("active");
  twoButton.classList.remove("active");
  threeButton.classList.remove("active");
}}

oneButton.addEventListener("click",()=>{
  console.log("one column");
  cleanContainer();
  removeActiveButton();
  oneButton.classList.add("active");
  gridContainer.classList.add("row-cols-1");
})
twoButton.addEventListener("click",()=>{
  console.log("two column");
  cleanContainer();
  removeActiveButton();
  twoButton.classList.add("active");
  gridContainer.classList.add("row-cols-2");
})
threeButton.addEventListener("click",()=>{
  console.log("three column");
  cleanContainer();
  removeActiveButton();

```

```

threeButton.classList.add("active");
gridContainer.classList.add("row-cols-3");
})

window.addEventListener("load", activateButton);
{% endblock %}

```

Текст шаблону profile.html:

```

{% extends "base.html" %}

{% block title %}
User {{ user }}
{% endblock %}

{% block content %}
<h5>This is {{ user.username }}'s profile.</h5>
<form action="{% url 'logout' %}" method="post">
  {% csrf_token %}
  <button class="btn btn-success mt-1" type="submit">Log out</button>
</form>

<div class="table-responsive">
<table class="table my-3">
<caption class="caption-top">Bunches processed: {{ bunches|length }}</caption>
<caption> is damaged and is good</caption>
<thead>
<tr>
<th>№</th>
<th>Number of grapes</th>
<th>Is outside damaged?</th>
<th>Is shape unstable?</th>
<th>Is color uneven?</th>
<th>Is sugar irregular?</th>
<th>Stage</th>
</tr>
</thead>
<tbody>
{% for b in bunches%}
<tr>
<td><a href="{% url 'grapes:bunch' b.id %}" class="text-success">{{ b.id }}</a></td>
<td>{{ b.count }}</td>
<td>{% if b.outside %} {% else %} {% endif %}</td>
<td>{% if b.shape %} {% else %} {% endif %}</td>
<td>{% if b.color %} {% else %} {% endif %}</td>
<td>{% if b.sugar %} {% else %} {% endif %}</td>
<td>{{ b.stage }}</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
<a class="btn btn-primary" href="{% url 'grapes:new_bunch' %}">Add new bunch</a>
{% endblock %}

```


Текст шаблона new_bunch.html:

```
{% extends 'base.html' %}
{% load crispy_forms_tags %}

{% block title %}
    New bunch
{% endblock %}

{% block content %}
<form method="post" class="form">
    {% csrf_token %}
    {{ form|crispy }}
    <button type="submit" class="btn btn-success">Add bunch</button>
</form>
{% endblock %}
```

Текст шаблона bunch.html:

```
{% extends "base.html" %}

{% block title %}
    Bunch {{ bunch.id }}
{% endblock %}

{% block content %}
<h3> Bunch #{{ bunch.id }}</h3>
<p>Has {{ bunch.count }} grapes</p>

<h5>Statistics</h5>
<ul class="list-group">
    <li class="list-group-item p-3">Outside {{ bunch.outside }}</li>
    <li class="list-group-item p-3">Shape {{ bunch.shape }}</li>
    <li class="list-group-item p-3">Color {{ bunch.color }}</li>
    <li class="list-group-item p-3">Sugar {{ bunch.sugar }}</li>
</ul>

<p class="fw-medium mt-3">Processed to stage: {{ bunch.stage }}</p>
<p><a class="text-success" href="{% url 'grapes:profile' %}">Bunch owner
profile</a></p>
{% endblock %}
```

Текст шаблона login.html:

```
{% extends 'base.html' %}
{% load crispy_forms_tags %}

{% block title %}
    Login
{% endblock %}

{% block content %}
<form method="post" class="form">
    {% csrf_token %}
    {{ form|crispy }}
    <button type="submit" class="btn btn-success">Log in</button>
```

```
</form>
{% endblock %}
```

Текст шаблону signup.html:

```
{% extends 'base.html' %}
{% load crispy_forms_tags %}

{% block title %}
Signup
{% endblock %}

{% block content %}
<form method="post" class="form">
  {% csrf_token %}
  {{ form|crispy }}
  <button type="submit" class="btn btn-success">Sign up</button>
</form>
{% endblock %}
```

Текст класу SignUpView:

```
class SignUpView(CreateView):
    form_class=UserCreationForm
    success_url=reverse_lazy('login')
    template_name='registration/signup.html'
```

Текст файлу accounts/urls.py:

```
urlpatterns = [
    path('signup/', SignUpView.as_view(), name='signup')
]
```

Текст файлу runner/urls.py:

```
urlpatterns = [
    path('', include('grapes.urls')),
    path('accounts/', include('accounts.urls')),
    path('accounts/', include('django.contrib.auth.urls')),
    path('admin/', admin.site.urls),
]
```

ДОДАТОК Г

Слайди презентації

Презентації курсової роботи

“Застосунок перевірки якості винограду”

Конструювання програмного забезпечення, 3 курс 2 семестр

прийняв Леощенко С.Д.
виконав Онищенко О.А. (КНТ-122)

Рисунок Г.1 - Слайд 1

Мета роботи

1

Набути досвід створення повноцінних
веб застосунків з фреймворком Django

Рисунок Г.2 - Слайд 2

Об'єкт розробки**2**

Розробка застосунку контролю якості
винограду

Рисунок Г.3 - Слайд 3

Задачі проєкту**3**

Створити застосунок, який взаємодіє з
базою даних для створення об'єктів.
Додати до нього інтерфейс. Зробити
його доступним на будь-якій
платформі, що має браузер.

Рисунок Г.4 - Слайд 4

Порівняння аналогів

4

Критерій	Clarifruit	Sectormentor	Grape Compass
Масштабованість	Середня	Висока	Низька
Робота без мережі	Є	Є	Немає
Зручність інтерфейсу	Середня	Висока	Низька
Можливість додавати фото	Є	Немає	Немає
Детальність статистики	Середня	Висока	Низька
Ціна	Немає	Висока	Немає

Рисунок Г.5 - Слайд 5

Порівняння мов програмування

5

Критерій	Django, Python	.NET, C#	Express, Node.js
Масштабованість	Середня	Висока	Низька
Легкість вивчення	Середня	Висока	Низька
Підтримка спільноти	Велика	Середня	Велика
Безпека	Висока	Висока	Середня

Рисунок Г.6 - Слайд 6

Порівняння середовищ розробки

7

Критерій	VS Code	Sublime Text	PyCharm
Зручність	Висока	Висока	Середня
Легкість	Середня	Висока	Низька
Кількість плагінів	Велика	Низька	Середня
Простота інтерфейсу	Середня	Висока	Середня

Рисунок Г.7 - Слайд 7

Функціональна схема

8

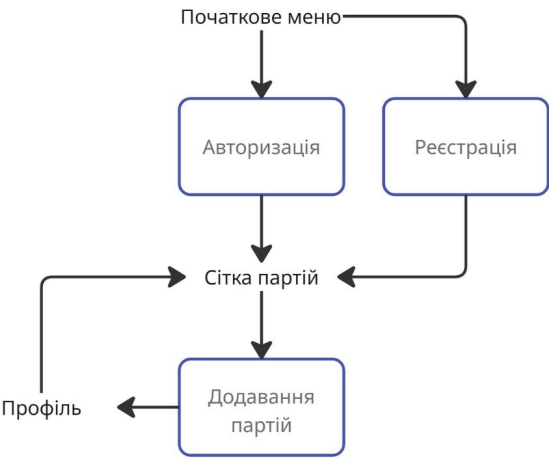


Рисунок Г.8 - Слайд 8

Функції системи**9**

- Реєстрація користувача
- Вхід користувача
- Вихід користувача
- Додавання партії
- Перегляд сітки партій
- Перегляд партії на її сторінці
- Перегляд партії в профілі

Рисунок Г.9 - Слайд 9

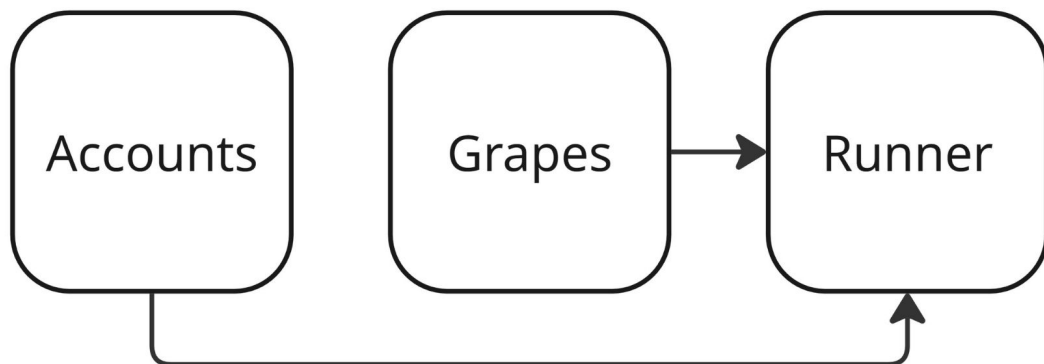
Проектування: структурна схема**10**

Рисунок Г.10 - Слайд 10

Проектування: шаблон

11

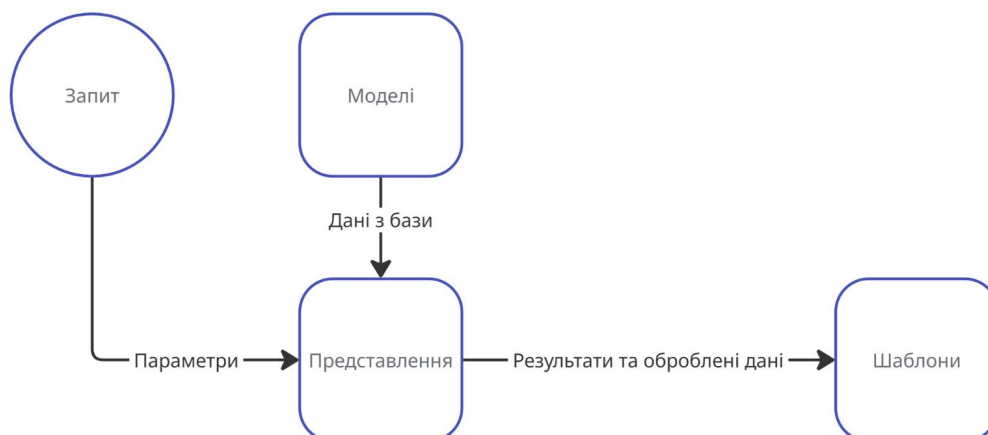


Рисунок Г.11 - Слайд 11

Проектування: модулі

12

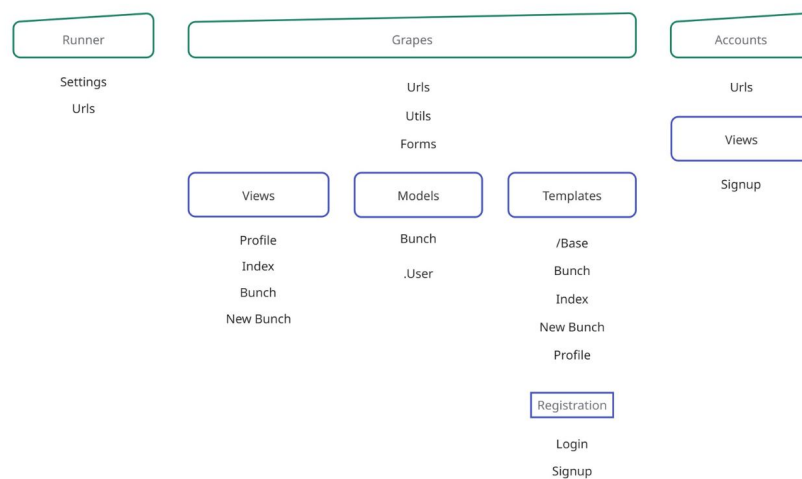


Рисунок Г.12 - Слайд 12

Концептуальна схема бази даних

13

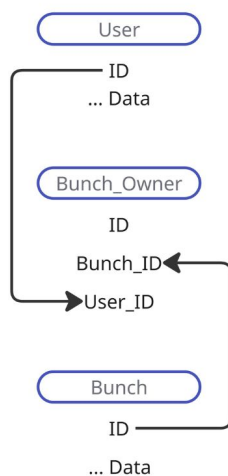


Рисунок Г.13 - Слайд 13

Приклад інтерфейсу

14

Grapes Checker

Profile

Add new bunch

Number of columns

One

Two

Three

Bunch 11 Stage: Jelly See results	Bunch 13 Stage: Jelly See results	Bunch 14 Stage: Jelly See results
Bunch 15 Stage: Juice	Bunch 22 Stage: Jelly	Bunch 23 Stage: Wine

Рисунок Г.14 - Слайд 14

Висновки**15**

Розроблено веб застосунок перевірки якості винограду. Всі функціональні вимоги виконано. Інтерфейс адаптується під різні екрани. Дані додаються і показуються коректно.

Рисунок Г.15 - Слайд 15